

Jenkins Practical Tasks – Automatic Build on Code Changes

Objective: Configure Jenkins so that changes pushed to a GitHub repository automatically trigger an application build.

Task 1 – Jenkins Poll SCM: Automatic Build on Code Change

Step 1: Create a GitHub Repository

Create a repository named jenkins-polling-demo. Add an index.html file and commit it to GitHub.

Step 2: Open Jenkins

Start Jenkins, log in, click New Item, enter Jenkins-Polling-Demo, select Freestyle project, and click OK.

Step 3: Configure GitHub

Under Source Code Management, select Git, enter the GitHub repository URL, configure credentials if required, and select */main as the branch.

Step 4: Configure Poll SCM

Under Build Triggers, select Poll SCM and enter * * * * * to check the repository every minute.

Step 5: Configure the Build

Under Build Steps, select Execute Windows batch command and enter:

```
echo Building Jenkins Application
echo Build Number: %BUILD_NUMBER%
echo Workspace: %WORKSPACE%
```

Step 6: Save and Build

Click Save, click Build Now, open Console Output, and verify that the build is successful.

Step 7: Test Automatic Build

Modify index.html in GitHub, commit the change, wait for Jenkins to poll the repository, and verify that a new build starts automatically.

Expected Flow

GitHub Code Change → Jenkins Poll SCM → Change Detected → Automatic Build → Build #2

Task 2 – Jenkins GitHub Webhook: Automatic Build

Step 1: Create GitHub Repository

Create jenkins-webhook-demo and add a simple index.html file. Commit and push it.

Step 2: Create Jenkins Job

Create a Freestyle project named Jenkins-Webhook-Demo.

Step 3: Configure Git

Under Source Code Management, select Git, enter the repository URL, configure credentials if needed, and set the branch to */main.

Step 4: Enable GitHub Trigger

Under Build Triggers, select GitHub hook trigger for GITScm polling.

Step 5: Add Build Command

Under Build Steps → Execute Windows batch command, enter:

```
echo =====  
echo Jenkins Automatic Build  
echo =====  
echo Build Number: %BUILD_NUMBER%  
echo Build completed successfully.
```

Step 6: Save

Click Save and run Build Now once to confirm Jenkins can access the repository.

Step 7: Configure GitHub Webhook

In GitHub, open Settings → Webhooks → Add webhook. Enter the Jenkins webhook URL, such as <http://YOUR-JENKINS-SERVER/github-webhook/>. Select application/json and Just the push event, then add the webhook.

Important

If Jenkins is running only on a local Windows computer, GitHub normally cannot reach localhost or a private LAN address directly. Use a publicly reachable Jenkins setup or institution-approved tunneling/network infrastructure.

Step 8: Test Webhook

Modify the application, commit and push the change using `git add .`, `git commit -m "Updated application"`, and `git push origin main`. GitHub should send the webhook and Jenkins should automatically start a build.

Step 9: Verify

Open Jenkins Build History and Console Output. Verify the new build number and a successful build message.

Expected Flow

Developer → Git Push → GitHub Webhook → Jenkins → Automatic Build → Success/Failure

Task 3 – Jenkins Pipeline: Checkout → Build → Test → Package

Step 1: Create Repository

Create jenkins-pipeline-demo and add the application source code and a Jenkinsfile.

Step 2: Create Jenkins Pipeline Job

Create a Jenkins Pipeline job named Jenkins-Pipeline-Demo.

Step 3: Create Jenkinsfile

Add the following Jenkinsfile:

```
pipeline {
  agent any
  stages {
    stage('Checkout') {
      steps { echo 'Checking out source code' }
    }
    stage('Build') {
      steps { echo 'Building application' }
    }
    stage('Test') {
      steps { echo 'Running test cases' }
    }
    stage('Package') {
      steps { echo 'Packaging application' }
    }
  }
  post {
    success { echo 'BUILD SUCCESSFUL' }
    failure { echo 'BUILD FAILED' }
  }
}
```

Step 4: Configure Pipeline

Set Definition to Pipeline script from SCM. Select Git, enter the GitHub repository URL, set Branch to */main, and Script Path to Jenkinsfile.

Step 5: Configure Automatic Trigger

Enable GitHub hook trigger for GITScm polling and configure the GitHub webhook.

Step 6: Build

Click Build Now and open Console Output. Verify Checkout, Build, Test, Package, and BUILD SUCCESSFUL messages.

Step 7: Modify Code

Change the application, commit and push the change. Jenkins should automatically execute the pipeline.

Step 8: Verify Pipeline

Verify the sequence: Checkout → Build → Test → Package → SUCCESS.

Task 4 – Automatic Build Number and Version

Step 1: Create Repository

Create jenkins-version-demo and add index.html.

Step 2: Create Jenkins Freestyle Project

Create Jenkins-Version-Demo as a Freestyle project and configure Git under Source Code Management.

Step 3: Configure Webhook

Under Build Triggers, select GitHub hook trigger for GITScm polling and configure the GitHub webhook.

Step 4: Add Build Command

Under Build Steps → Execute Windows batch command, enter:

```
echo =====  
echo Jenkins Application Build  
echo =====  
echo Build Number: %BUILD_NUMBER%  
echo Job Name: %JOB_NAME%  
echo Build ID: %BUILD_ID%  
echo =====
```

Step 5: Perform First Build

Click Build Now. Jenkins should create Build #1.

Step 6: Change Application

Modify the application and push the change. Jenkins should automatically create Build #2. Make another change and push again to create Build #3.

Step 7: Verify

Demonstrate that each code change results in a new Jenkins build number.